



TENANTLY

*A Mobile Property Management System for Small Landlords
and Tenants*

by

Avalbir Singh Banga (2440114)

Under the guidance of

Dr. Logeshwaran J

A Project report submitted in partial fulfilment of the requirements
of

V Semester B.Sc. (Computer and Mathematics)

CHRIST (Deemed to be University)

August 2026



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

CERTIFICATE

*This is to certify that the report titled “**Tenantly: A Mobile Property Management System for Small Landlords and Tenants**” is a bona fide record of work done by **Avalbir Singh Banga (2440114)** of CHRIST (Deemed to be University), Bangalore, in partial fulfilment of the requirements of V Semester BSc-CM during the year 2026-2027.*

Head of Department
Dr. Rupali Sunil Wagh

Project Guide
Dr. Logeshwaran J

Valued by:

- Name** : Avalbir Singh Banga
Register Number : 2440114
- Examination Centre** : CHRIST (Deemed to be University)
Date of Exam : 19/08/2026

AI Usage Declaration Statement

I hereby declare that Artificial Intelligence tools were used only for supportive purposes during the development and documentation of this project. AI assistance was used for activities such as idea exploration, conceptual clarification, code suggestions, debugging assistance, structural improvements, documentation support, and grammar refinement.

All project requirements, system design decisions, implementation, testing, verification, and final evaluation of the Tenantly – A Mobile Property Management System for Small Landlords and Tenants project were reviewed and understood by me. AI-generated suggestions were not accepted blindly and were checked, modified, and integrated based on the actual requirements and implementation of the project. I take full responsibility for the authenticity, correctness, and academic integrity of the submitted work.

The AI tools used in this project and their purposes are listed below:

AI tools used and purpose of use:

- OpenAI Codex – used for programming assistance, code suggestions, debugging, identifying implementation issues, and improving parts of the application code and project structure.
- Google Antigravity – used for development assistance, code generation and refinement and troubleshooting

The above AI tools were used as assistive development tools. All generated or suggested material was reviewed and adapted according to the requirements and actual implementation of the Tenantly project.

Name of the Student : Avalbir Singh Banga

Register Number : 2440114

Project Title : A Mobile Property Management System for Small Landlords

Signature :

Date : 19/08/2026

ACKNOWLEDGMENTS

We express our sincere thanks to all faculty members and staff of the Department of Computer Science, CHRIST (Deemed to be University), for their valuable suggestions during the course of this project.

First of all, we thank God Almighty for His immense grace and blessings showered on us at every stage of this work. We are grateful to our respected Head of the Department, Department of Computer Science, CHRIST (Deemed to be University), **Dr. Rupali Sunil Wagh**, for providing us the opportunity to take up this project as part of our curriculum, and Associate HOD, Department of Computer Science, CHRIST (Deemed to be University), **Dr. Gobi R**, for supporting us in all project-related activities.

We also express our gratitude to the Coordinator, Department of Computer Science, CHRIST (Deemed to be University), **Dr. Vaidhehi V**, for her support throughout the project.

We are grateful to our project in-charge and guide, Assistant Professor **Dr. Logeshwaran J**, Department of Computer Science, CHRIST (Deemed to be University), who was always there to support us throughout the project and was always present whenever we needed guidance and assistance. We sincerely thank him for his continuous support and presence whenever needed.

We would also like to thank our Alumni evaluator, whose knowledge and guidance benefited us in making the necessary changes in accordance with industry requirements. Thank you for your valuable support.

Last but not least, we would like to thank everyone who was involved in the project, directly or indirectly, for their support and contribution

Abstract

Tenantly was developed as a cross-platform mobile property-management system for small landlords, paying guests, hostel operators and tenants in India. The project addressed the operational fragmentation created by spreadsheets, handwritten registers and disconnected messaging channels. The implemented application consolidated property inventory, residents, tenancy assignments, invoices, manual payment verification, receipts, complaints, notices, documents and operational reporting into role-aware workflows. The client application was built with Expo, React Native and TypeScript, while Supabase provided authentication, PostgreSQL persistence, Row Level Security, private storage and database functions. The design deliberately treats organization membership as an authorization boundary and places financial and occupancy mutations inside transactional database operations. This reduces the risk of duplicate invoices, overlapping occupancy assignments and unauthorized payment decisions. The project was verified through TypeScript compilation, linting and a Jest suite comprising forty-five passing tests across seventeen suites. The report documents the requirements, architecture, schema, implementation approach, security model and test evidence. Tenantly demonstrates that a mobile-first system can give small property operators a clearer operational view while preserving tenant privacy and auditable financial records.

Keywords: property management, React Native, Expo, Supabase, Row Level Security, tenancy, invoices, mobile application.

Table of Contents

CHAPTER 1	1
1.1 Overview of Tenantly	1
1.2 Background and Motivation	1
1.3 Problem Statement	1
1.4 Objectives	1
1.5 Scope and Applicability	2
1.6 Intended Users	2
1.7 Organization of the Report.....	2
CHAPTER 2	3
2.1 Existing Property-Management Practices.....	3
2.2 Limitations of the Existing Process	3
2.3 Literature Review.....	3
2.4 Proposed System.....	5
2.5 Benefits of Tenantly.....	5
2.6 Functional Requirements	6
2.7 Non-Functional Requirements	6
2.8 Software and Hardware Requirements	7
2.9 Feasibility Analysis.....	8
CHAPTER 3	9
3.1 Overall Architecture.....	9
3.2 Role and Use-Case Design.....	9
3.3 Module Design.....	10
3.4 Data-Flow Design	10

3.5 Entity-Relationship Design	11
3.6 Core Database Entities	12
3.7 Authorization and Security Design	14
3.8 Workflow and Navigation Design	15
3.9 Authentication and Role-Routing Flow	17
3.10 Property, Resident and Tenancy Lifecycle	18
3.11 Invoice, Payment and Receipt Flow	19
3.12 Complaint Resolution and Operational Follow-up	20
CHAPTER 4	21
4.1 Development Methodology	21
4.2 Technology Stack.....	21
4.3 Authentication and Protected Navigation	21
4.4 Organization, Property and Occupancy Management	21
4.5 Billing, Payments and Receipts	22
4.6 Complaints, Notices, Documents and Operations	22
4.7 Dashboards, Reports and Shared Components	22
4.8 Database Migrations and RLS	22
4.9 Validation, Errors and Reliability	22
4.10 Authentication and Session Implementation	23
4.11 Property and Occupancy Implementation Details	23
4.12 Billing and Payment Details	24
4.13 Complaints, Notices and Documents	24
4.14 Reliability and Data-Integrity Implementation	24
CHAPTER 5	25
5.1 Testing Objectives and Environment.....	25

5.2 Unit and Component Testing	25
5.3 Static Analysis	25
5.4 Database and RLS Test Readiness	26
5.5 Expo Environment Check	26
5.6 Test Execution Summary	26
5.7 Representative Test Cases.....	27
5.8 Defect and Risk Resolution	29
5.9 Extended Functional and Security Verification Matrix	31
CHAPTER 6	32
6.1 Summary of Implementation	32
6.2 Achievement of Objectives.....	33
6.3 Advantages.....	33
6.4 Current Limitations	33
6.5 Future Scope	33
Appendix A: Installation and Configuration.....	35
Configuration Checklist	35
Appendix B: Basic User Manual	36
Owner and Manager Workflow	36
Tenant Workflow	36
Maintenance Staff Workflow.....	36
Appendix C: Important Database Functions.....	38
Data Integrity Controls	39
Appendix D: Extended Test-Case Matrix.....	40
Appendix D Continued: Extended Test-Case Matrix	42

Appendix E: Verification Commands.....	44
Appendix F: Role and Data-Access Matrix	45
Appendix G: Data Model Traceability	46

LIST OF FIGURES

- Figure 3.1. Tenantly system architecture
- Figure 3.2. Role-aware use-case model
- Figure 3.3. Monthly invoice and payment workflow
- Figure 3.4. Core data model
- Figure 3.5. Complaint resolution workflow
- Figure 3.6. Role-aware navigation model
- Figure 3.7. Authentication and role-routing flow
- Figure 3.8. Property, resident and tenancy lifecycle
- Figure 3.9. Invoice, payment and receipt flow
- Figure 3.10. Complaint resolution and operational follow-up

LIST OF TABLES

- Table 2.1. Functional requirements
- Table 2.2. Non-functional requirements
- Table 2.3. Software and hardware requirements
- Table 3.1. Core data entities
- Table 5.1. Test execution summary
- Table 5.2. Representative test cases
- Table 5.3. Extended verification matrix - identity, occupancy and finance
- Table 5.4. Extended verification matrix - operations, validation and tooling
- Table F.1. Role and data-access summary
- Table G.1. Major entity groups and their design rationale

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CSV	Comma-Separated Values
RLS	Row Level Security
RPC	Remote Procedure Call
SDK	Software Development Kit
UI	User Interface
UUID	Universally Unique Identifier

CHAPTER 1

INTRODUCTION

1.1 Overview of Tenantly

Tenantly is a mobile-first property-management application for small landlords, paying-guest operators, hostel managers, tenants and maintenance staff. The implemented system is a private operational platform rather than a public rental marketplace. It combines property inventory, resident onboarding, tenancy and occupancy history, invoicing, manual payment verification, receipts, complaints, notices, protected documents and reporting within role-aware application areas.

1.2 Background and Motivation

Small property operators frequently manage rent, occupancy and maintenance through paper registers, spreadsheet files and messaging applications. These methods are flexible at a very small scale but become difficult to audit when a property contains multiple rooms, beds, residents and billing periods. Tenantly was motivated by the need for an accessible mobile workflow that reduces duplicate entries, makes current occupancy visible and preserves a traceable record of financial and operational decisions.

1.3 Problem Statement

The project addresses the absence of a single, accessible system for small-property operations and tenant self-service while preserving isolation between organizations. Fragmented records do not reliably represent room transfers, partial payments, approval decisions or complaint histories. Tenantly therefore requires a consistent data model, protected workflows and database-enforced authorization so that each record remains traceable without exposing another organization's data.

1.4 Objectives

The objectives were to implement secure authentication and organization membership; manage properties, rooms, beds, residents and tenancies; preserve

occupancy history; generate monthly invoices idempotently; record and decide manual payments; retain receipt evidence; manage complaints, notices and private documents; and provide role-specific dashboards, reports and exports. Reliability, privacy and auditability were treated as core implementation objectives rather than interface-only concerns.

1.5 Scope and Applicability

The implemented MVP supports small residential properties, paying-guest accommodation and hostel-like operations. It covers owner, manager, tenant and maintenance-staff workflows inside an organization and deliberately prioritizes auditable manual payment recording over payment-gateway integration. Public property discovery, online payment processing and marketplace functionality remain outside the implemented scope.

1.6 Intended Users

Owners and managers use Tenantly to configure operations, review finance and handle resident requests. Tenants use the application to view invoices, submit payment proof, access notices and create complaints. Maintenance staff receive and update assigned work. This separation makes the user interface focused while the database maintains the authorization boundary.

1.7 Organization of the Report

The remaining chapters describe system analysis and requirements, the proposed architecture and database design, the implementation of the application and its security controls, verification evidence, and a conclusion with clearly separated future enhancements.

CHAPTER 2

SYSTEM ANALYSIS AND REQUIREMENTS

2.1 Existing Property-Management Practices

The existing process generally distributes information across spreadsheets, paper documents and message threads. A room may be available in one register while a separate chat records an informal booking. Rent entries can be updated independently of occupancy changes, which makes reconciliation difficult. The process also provides limited tenant self-service because residents must contact an owner for routine information such as outstanding rent or the latest notice.

2.2 Limitations of the Existing Process

Manual and disconnected approaches create duplicate data entry, unclear ownership of records, delayed communication and weak auditability. They do not reliably enforce exclusivity when assigning beds, provide no consistent workflow for payment approval, and rarely isolate information by property organization. Tenantly addresses these gaps through a single backend model, transaction-oriented operations and role-based navigation.

2.3 Literature Review

The design is grounded in established information-system practices: normalized relational data for operational consistency, database authorization for controlled sharing, transactional operations for multi-record financial and occupancy changes, and mobile interaction patterns for field use. Official Expo, React Native, Supabase and PostgreSQL documentation informed the technology and security choices. These sources support the architectural approach; repository code, migrations and executed checks provide the evidence for what Tenantly actually implements.

2.3.1 Context of Small-Property Management

Property management software is often discussed in the context of large apartment portfolios, commercial facilities or marketplace platforms. Tenantly addresses a narrower operational setting: small landlords, paying-guest accommodation and hostel-

like properties where a limited administrative team may handle occupancy, rent collection, resident communication and maintenance at the same time. In this environment, the main difficulty is not the absence of individual tools. Spreadsheets can record rent, messaging applications can distribute notices and cloud folders can hold documents. The difficulty arises because these tools represent related facts independently. A resident may appear in one spreadsheet, a payment may be discussed in a message thread and a complaint may be tracked informally without a common record connecting them. Tenantly therefore treats consolidation and traceability as the central design problem rather than attempting to reproduce the broad feature set of an enterprise property-management suite.

The project also recognizes that small-property operations contain recurring relationships that are naturally represented as structured data. A property contains rooms and, where required, beds; a resident enters a tenancy; a tenancy produces occupancy history and periodic invoices; payments are associated with those invoices; and residents generate operational interactions such as complaints and document requests. A relational model is suitable for this setting because these entities have clear identities and dependencies. The design preserves these relationships so that a change in one workflow can be interpreted in the context of the others instead of being recorded as an isolated note. This is particularly important for vacancy status and financial records, where an inaccurate or duplicated entry can affect later decisions.

2.3.2 Comparison with Manual and Fragmented Workflows

A manual workflow is flexible because it imposes very little initial setup. A landlord can begin with a notebook, a spreadsheet and a messaging group, and each tool can be adapted immediately. However, the same flexibility creates inconsistency as the number of residents and billing periods grows. Data must be copied between files, names and room numbers may be entered differently, and there is no automatic assurance that an old occupancy record has been closed before a new one is created. Financial verification is also difficult to audit when payment evidence is sent through private messages and approval is communicated separately. Tenantly replaces these disconnected actions with linked records and explicit state changes.

The proposed system does not assume that every payment is processed through an online gateway. Instead, the implemented workflow deliberately supports manual

payment evidence and an approval or rejection decision. This choice reflects the project scope while still improving auditability. The payment itself may originate outside the application, but the evidence, decision and allocation are represented within the system. Likewise, complaints and notices are not treated as generic messages. They are structured records with states, recipients or assignments, which makes their progress visible to authorized users. The design therefore improves control without requiring every real-world activity to be automated.

2.3.3 Design Principles Derived from the Review

Four principles guided the design. First, authorization is enforced at the data layer because route visibility alone cannot protect multi-organization records. Second, financial and occupancy changes use transaction-oriented database functions when several records must change together. Third, retry-sensitive operations use idempotency or uniqueness controls. Fourth, each role receives focused navigation while all roles remain subject to the same backend policies. These principles connect the review directly to the implemented system.

2.4 Proposed System

Tenantly is implemented as an Expo and React Native client backed exclusively by Supabase. Supabase Auth establishes identity; PostgreSQL stores organization-scoped operational data; Row Level Security and protected functions enforce access and state transitions; and private Storage buckets retain payment proof, complaint attachments, resident documents and expense receipts. The client uses direct RLS-protected queries for suitable CRUD operations and RPCs for transactions such as tenancy assignment, payment decisions and invoice generation.

2.5 Benefits of Tenantly

The system centralizes operational data, reduces repeated manual calculations, enables tenant self-service, captures approval history and improves visibility of vacancies and outstanding rent. The use of organization-scoped records and RLS policies supports privacy across property operators. The mobile-first experience also makes routine updates practical during on-site property work.

2.6 Functional Requirements

Table 2.1. Functional requirements.

ID	Requirement	Implemented evidence
FR-01	Users can register, sign in, recover credentials and enter protected routes.	Authentication routes, callback handling and session-aware root layout.
FR-02	Owners can create organizations, properties, rooms, beds and residents.	Organization, property, room and resident setup routes and services.
FR-03	Tenancies preserve assignment history and reject conflicting occupancy.	create_tenancy_with_assignment, transfer_occupancy and availability checks.
FR-04	Owners can generate invoices and manage manual payments.	Billing services, rent/invoice routes and protected finance RPCs.
FR-05	Tenants submit proof; authorized operators decide payments.	Tenant payment-proof route, owner decision route, RLS and allocation logic.
FR-06	Users can create, assign, view and transition complaints.	Owner/tenant complaint routes, staff tasks, events and transition RPCs.
FR-07	Operators publish targeted notices; recipients maintain read state.	Notice service, targets, read records and protected policies.
FR-08	Users access role-specific dashboards, reports and exports.	Owner, tenant and staff route groups; profit report and CSV export.

2.7 Non-Functional Requirements

Table 2.2. Non-functional requirements.

Category	Requirement	Design response
Security	Organizations must not access one another's records.	RLS policies and organization IDs on business records.
Integrity	Financial and occupancy operations must be auditable.	Database functions, constraints, allocations and receipt immutability.
Usability	First-time owners should complete setup without training.	Dedicated setup screens and focused navigation.
Reliability	Repeated mobile requests must not create duplicate results.	Idempotent invoice generation and transaction-oriented mutations.
Accessibility	The interface must support clear labels and usable targets.	Shared component system and platform-aware controls.
Maintainability	The system should separate domain logic from routes.	Feature-oriented services, shared utilities and generated database types.

2.8 Software and Hardware Requirements

Table 2.3. Software and hardware requirements.

Component	Requirement
Development environment	Node.js, npm, Expo tooling and Supabase CLI
Mobile client	Expo SDK 56 application configured for Android and iOS through EAS profiles
Web client	Expo Router / React Native Web export supported by the shared client
Backend	Supabase Auth, PostgreSQL, private Storage and database functions
Recommended workstation	Modern multi-core CPU, 8 GB RAM, internet access and Docker for local Supabase
Verification	Jest, TypeScript, Expo ESLint, Expo Doctor and Supabase pgTAP

2.9 Feasibility Analysis

Technical feasibility is supported by a shared TypeScript client for Android, iOS and web, a versioned Supabase backend and repository-based verification tooling. Operational feasibility comes from task-focused owner, tenant and staff routes and from manual payment verification that matches common small-property practice. Economic feasibility benefits from one client codebase and managed backend services. Exact mobile deployment baselines are determined by the Expo/EAS build configuration rather than asserted independently in this report.

CHAPTER 3

SYSTEM DESIGN

3.1 Overall Architecture

Tenantly uses a layered client-and-backend architecture. Expo Router provides public, setup, owner, tenant and staff route groups; feature services coordinate Supabase queries and RPCs; TanStack Query manages remote state and scoped invalidation; and shared components provide consistent loading, error and navigation behavior. Supabase Auth supplies identity, PostgreSQL stores business records, RLS evaluates access, private Storage holds sensitive files, and database functions perform atomic domain operations.

Tenantly system architecture

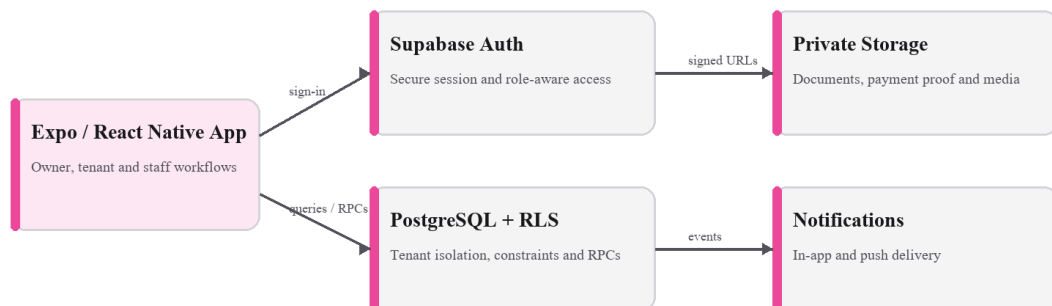


Figure 3.1. Tenantly system architecture.

3.2 Role and Use-Case Design

The role model separates interface responsibilities without treating navigation as authorization. Owners administer an organization; managers operate only within granted scope; tenants access records connected to their own resident or tenancy relationship; and maintenance staff receive task-focused access. Organization and

property memberships, RLS policies and protected functions remain the authoritative boundary for every role.

Role-aware use-case model

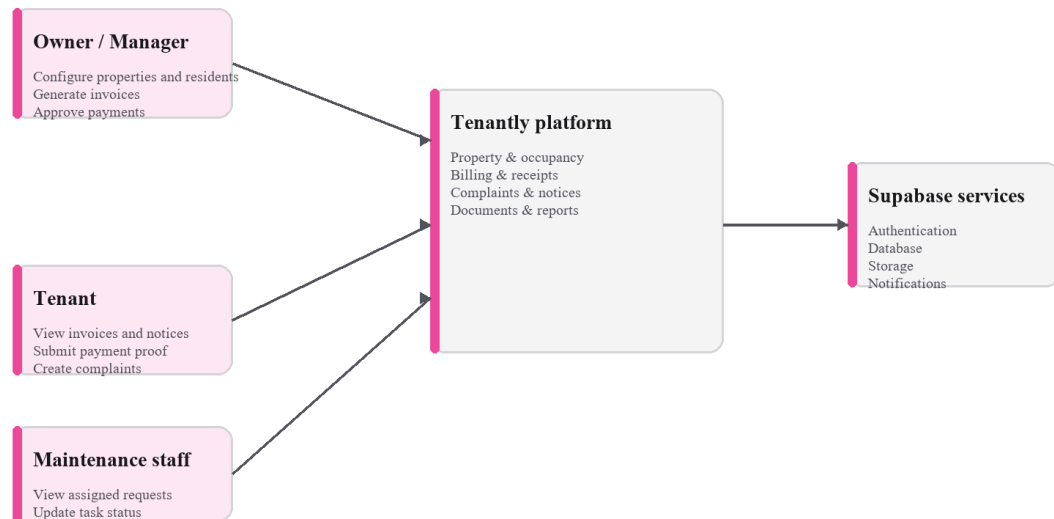


Figure 3.2. Role-aware use-case model.

3.3 Module Design

The repository is organized around authentication, organizations, properties, residents, billing, complaints, notices, operations, notifications and shared infrastructure. Expo Router contains distinct owner, tenant and staff screens, while feature services encapsulate queries, validation and RPC calls. Shared query keys, error translation, money/date/CSV utilities, theme tokens and reusable state components reduce duplicated behavior without hiding domain rules inside generic route code.

3.4 Data-Flow Design

For a protected action, the client supplies the authenticated session and active organization context to an RLS-protected query or approved RPC. PostgreSQL then validates membership, record ownership, domain state and constraints before committing the change. On success, TanStack Query invalidates only the affected scoped keys; on failure, the application maps the backend error to user-facing feedback. High-risk finance operations avoid presenting an optimistic result before the backend confirms it.

Monthly invoice and payment workflow

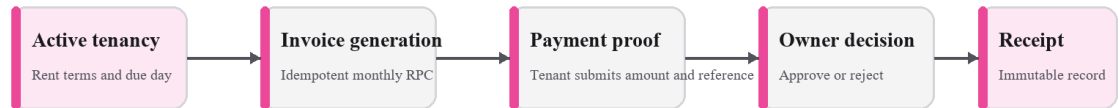


Figure 3.3. Monthly invoice and payment workflow.

3.5 Entity-Relationship Design

The schema separates identity, organization membership, physical inventory, contractual tenancy, time-aware occupancy, billing, operational records and protected files. Business tables carry organization scope where applicable. Residents remain independent of authentication; occupancy assignments preserve transfers; invoices and payment allocations separate amounts due from approved money; and complaints retain event history. This decomposition enables meaningful foreign keys, uniqueness constraints and RLS policies.

Core data model

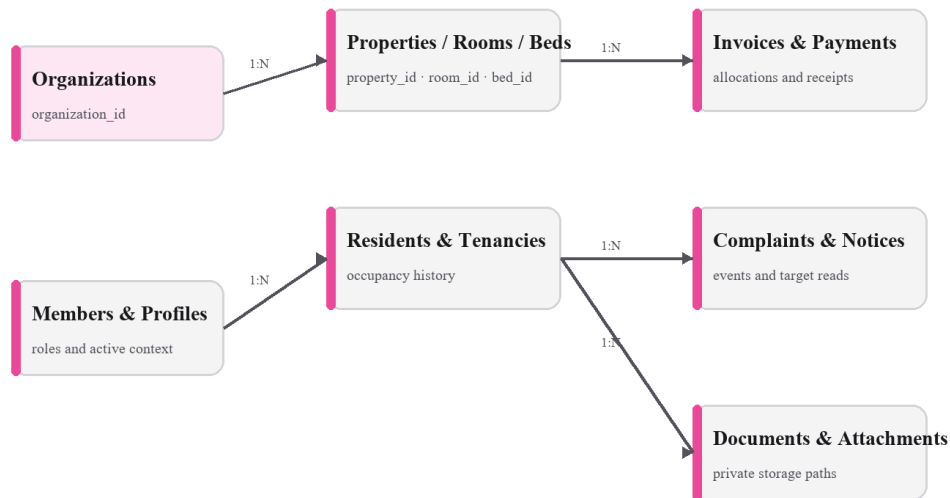


Figure 3.4. Core data model.

3.6 Core Database Entities

Table 3.1. Core data entities.

Entity group	Purpose
Identity and membership	Profiles, organizations, organization memberships, property memberships and invitations.
Property and occupancy	Properties, rooms, beds, residents, tenancies and occupancy assignments.
Billing	Invoices, invoice items, payments, payment allocations and receipts.
Operations	Complaints, complaint events, notices, notice targets, documents and notifications.
Audit and storage	Audit logs, attachment metadata, push devices and private object paths.

3.6.1 Organization and Membership Model

The organization is the principal administrative boundary in Tenantly. Business records are associated with an organization so that ownership remains explicit throughout queries and policies. A user profile alone does not grant broad access to property data; access is interpreted through membership and role information. This structure supports the core requirement that two unrelated property operators must not be able to read or modify one another's residents, tenancy records, invoices or operational documents. It

also allows the application to present different navigation areas while relying on the same underlying authorization model.

Membership records connect authenticated profiles to an organization and, where applicable, selected properties. Invitations create that relationship through a single-use token and an intended role instead of trusting a client-supplied organization identifier. The current migration sequence also includes a focused email-validation correction for `create_invitation`, while the client validates trimmed email input and role/property requirements before invoking the RPC.

3.6.2 Property, Room, Resident and Tenancy Relationships

The property and occupancy model separates physical inventory from contractual occupancy. Properties contain rooms and may contain individual beds when shared accommodation requires bed-level assignment. Residents are represented independently of rooms so that a person can exist in the system before a tenancy begins and can retain historical relationships after moving out. A tenancy records the contractual relationship, while occupancy assignments represent where the resident is placed during particular periods. This separation avoids overwriting history when a resident transfers between rooms or beds.

Preserving assignment history is important for both operational reporting and dispute resolution. If the current room were stored only as a field on the resident record, every transfer would destroy the previous location. Tenantly instead closes the earlier active assignment and creates a new assignment through guarded database operations. The design can therefore derive current occupancy while retaining past states. Constraints and database functions are used to reduce the risk of conflicting active assignments, which is more reliable than depending only on a client form to check whether a room or bed appears available.

3.6.3 Billing, Payment and Receipt Relationships

Billing is modeled as a sequence of related but distinct records. An invoice represents an amount due for a billing period and can contain line items. A payment represents submitted financial evidence and its decision state. Payment allocations connect approved amounts to invoices, and a receipt represents completed financial evidence intended to remain immutable. Keeping these concepts separate allows the system to

represent partial or multiple payments without rewriting the original invoice. It also creates a clearer audit trail because submission, approval and allocation are not collapsed into a single editable value.

Monthly invoice generation is designed to be idempotent for eligible tenancies. This matters in a mobile system because a user may repeat an action after a slow connection or uncertain response. A duplicate monthly invoice would create a serious financial inconsistency, so the database operation and uniqueness rules are designed to make repeated requests safer. Payment decision logic is similarly placed behind an authorized server-side/database operation, where the approved amount and allocation rules can be checked together before records are committed.

3.6.4 Operational Records and Auditability

Complaints, notices, documents, notifications, audit logs and expenses form the operational layer. Complaints preserve assignments and events; notices separate publication, targeting and recipient read state; and files are represented by protected object paths rather than permanent public URLs. Push registration is implemented but activates only when `EXPO_PUBLIC_EAS_PROJECT_ID` is configured, while Sentry initialization remains optional through `EXPO_PUBLIC_SENTRY_DSN`.

3.7 Authorization and Security Design

Authorization is enforced through PostgreSQL Row Level Security, membership helper functions and protected RPCs. Policies evaluate organization, property, resident or payment scope rather than inferring access from the visible screen. The mobile configuration exposes only public Supabase connection values; service-role credentials are excluded. Storage buckets are private and apply record-aware read policies plus controlled upload paths.

3.7.1 Row Level Security as the Primary Data Boundary

Tenantly uses Row Level Security as a database-level authorization mechanism. The purpose is to ensure that a request must satisfy the relevant membership or tenant-scope rule even if the client application is modified, a route is called directly or an unexpected navigation state occurs. Client-side role checks still improve usability by hiding irrelevant actions, but they are not treated as the final security boundary. This division of responsibility follows the principle that the interface may suggest what a user can

do, while the backend must decide whether the requested data operation is actually permitted.

Organization identifiers on business records make policy conditions explicit. Owner and manager operations are evaluated against authorized memberships, while tenant-facing records are restricted to the tenancy or resident relationships that belong to the authenticated user. Maintenance access is intentionally narrower and focuses on assigned operational work. This approach supports least-privilege behavior across the main roles and reduces dependence on a single broad application role.

3.7.2 Integrity Controls and Protected Operations

Authorization alone does not prevent inconsistent data, so Tenantly combines policies with relational constraints and database functions. Primary and foreign keys preserve entity relationships; uniqueness rules help prevent duplicate business states; and guarded state transitions restrict changes that should occur only through approved workflows. Multi-row operations such as tenancy assignment, occupancy transfer, invoice generation and payment decisions are handled transactionally so that dependent records are not left partially updated when an operation fails.

The same principle applies to stored files and financial evidence. Private storage prevents sensitive attachments from being treated as public assets, and the database retains protected paths that can be resolved only for authorized access. Completed receipts are designed as immutable records rather than ordinary editable notes. Together, these choices support confidentiality, integrity and traceability without assuming that the mobile client itself is trusted with privileged credentials.

3.8 Workflow and Navigation Design

Complaint resolution workflow

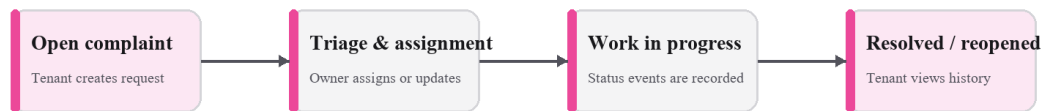


Figure 3.5. Complaint resolution workflow.

Role-aware navigation

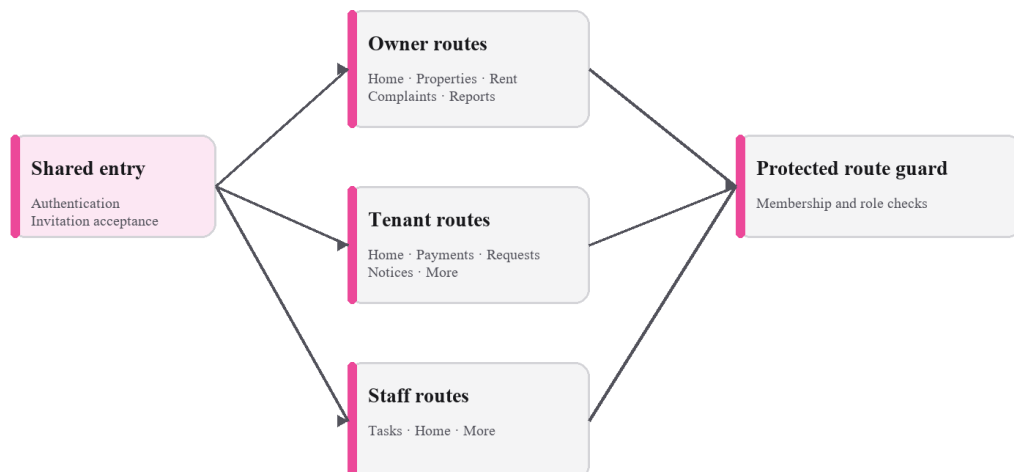


Figure 3.6. Role-aware navigation model.

3.9 Authentication and Role-Routing Flow

Authentication in Tenantly is designed as a sequence in which identity is established before any role-specific application area is exposed. The client restores the Supabase session, checks whether the user has the required organization or invitation context, and then directs the user to the appropriate protected route group. This flow improves usability, but it does not replace database authorization: every protected data request remains subject to Row Level Security and membership checks.

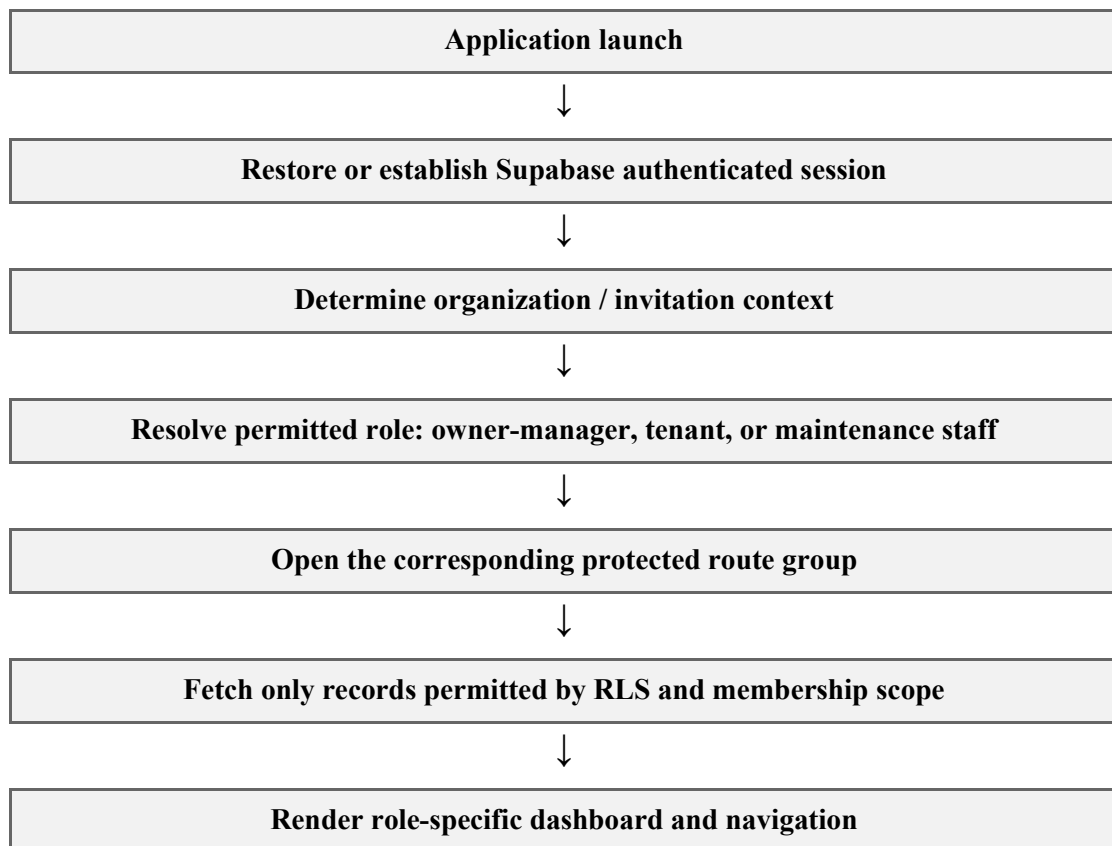


Figure 3.7. Authentication and role-routing flow.

The separation between navigation and authorization is deliberate. A hidden screen is not treated as a security control. Even if a user attempts to reach a route directly, the database policy layer still evaluates whether the authenticated identity is permitted to read or mutate the requested records. This reduces the risk of accidental cross-organization exposure and makes the backend the authoritative security boundary.

3.10 Property, Resident and Tenancy Lifecycle

Property management is organized around a stable inventory hierarchy and a separate occupancy history. Owners or managers first configure the organization and property inventory, then create resident records and tenancy terms. Occupancy is assigned only after the required entities exist, allowing physical inventory, contractual terms and historical assignments to remain distinguishable in the database.

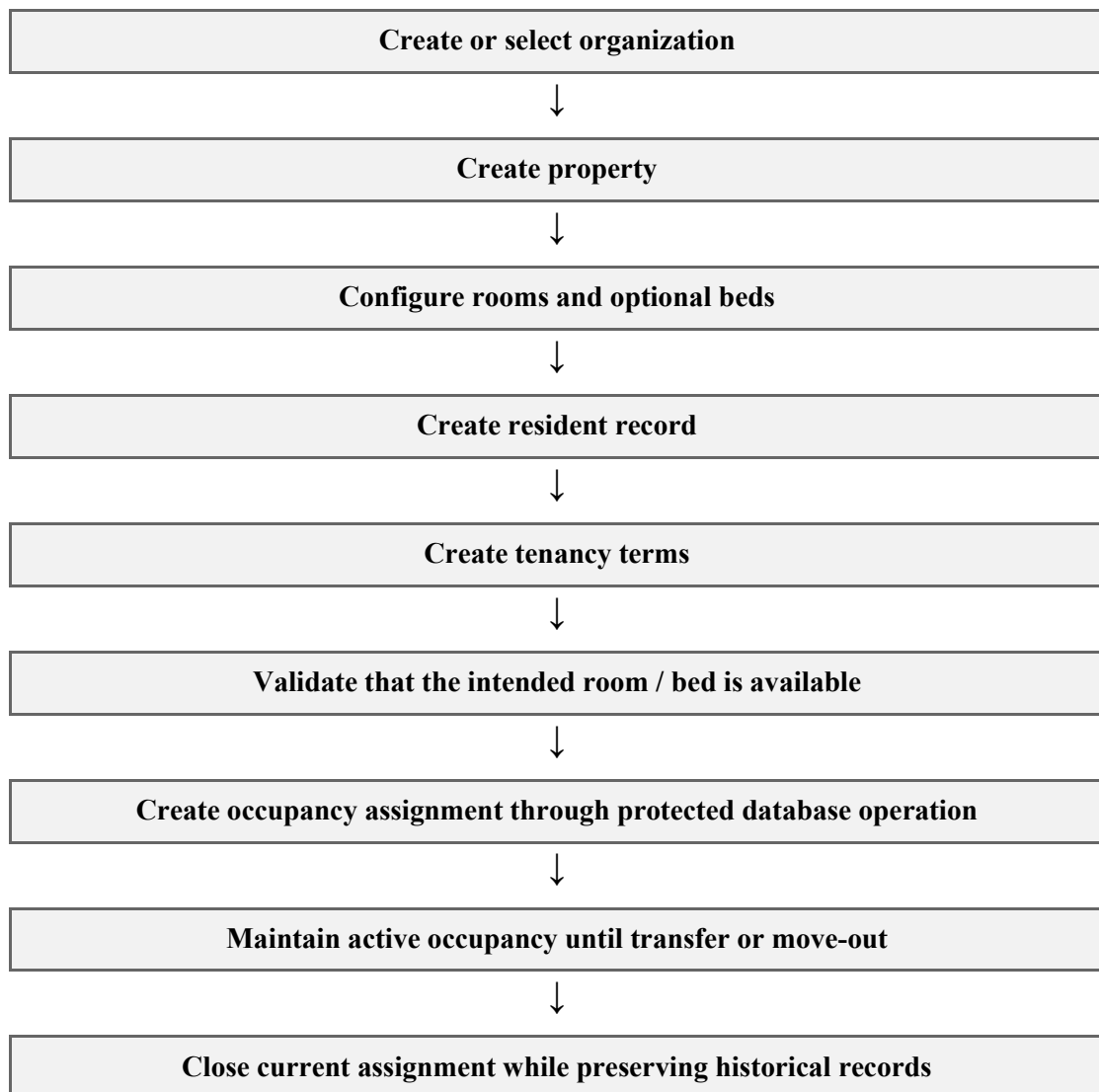


Figure 3.8. Property, resident and tenancy lifecycle.

Preserving historical assignments is more reliable than overwriting a single current-room field. It allows the system to retain evidence of transfers and move-outs while still deriving present vacancy from active assignments. The transactional tenancy and transfer operations described in the implementation also reduce the chance of two active assignments being created for the same constrained space.

3.11 Invoice, Payment and Receipt Flow

The financial workflow separates billing, payment evidence, approval and receipt generation into different records. Monthly invoices are generated for eligible tenancies, after which a tenant can review the amount due and submit manual payment information with supporting evidence. An authorized operator then decides the payment, and approved amounts are allocated without exceeding the accepted value.

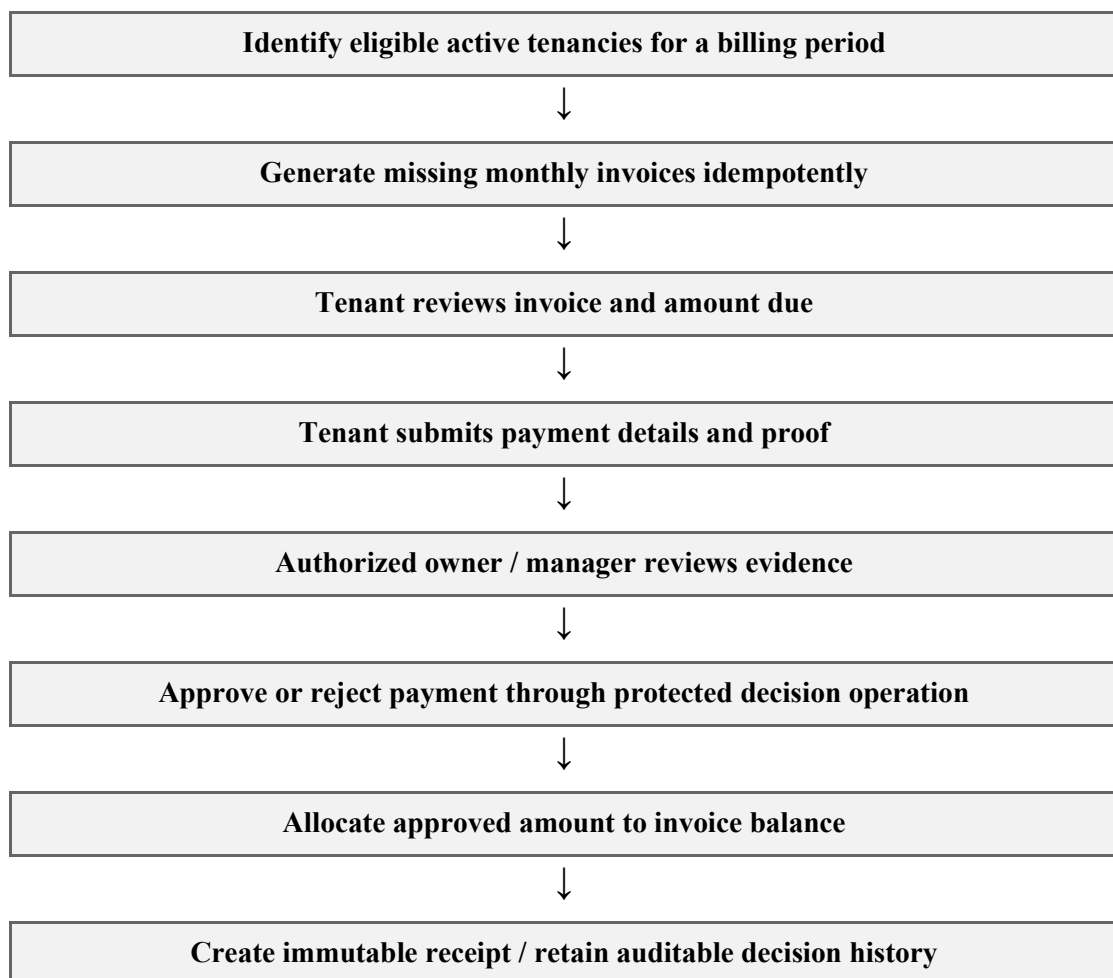


Figure 3.9. Invoice, payment and receipt flow.

This design is intentionally conservative because financial records require traceability. Idempotent generation prevents repeated mobile requests from producing duplicate invoices, while the protected payment-decision operation keeps allocation rules in the backend. Receipt immutability and retained payment evidence provide a clearer audit trail than directly editing an invoice balance.

3.12 Complaint Resolution and Operational Follow-up

Complaint management is modeled as a lifecycle rather than a single editable status value. A tenant creates a request with descriptive information; an authorized operator can triage or assign it; maintenance staff receive only appropriately scoped tasks; and allowed state changes are recorded through the complaint workflow. This creates a chronological operational history that can be reviewed after resolution.

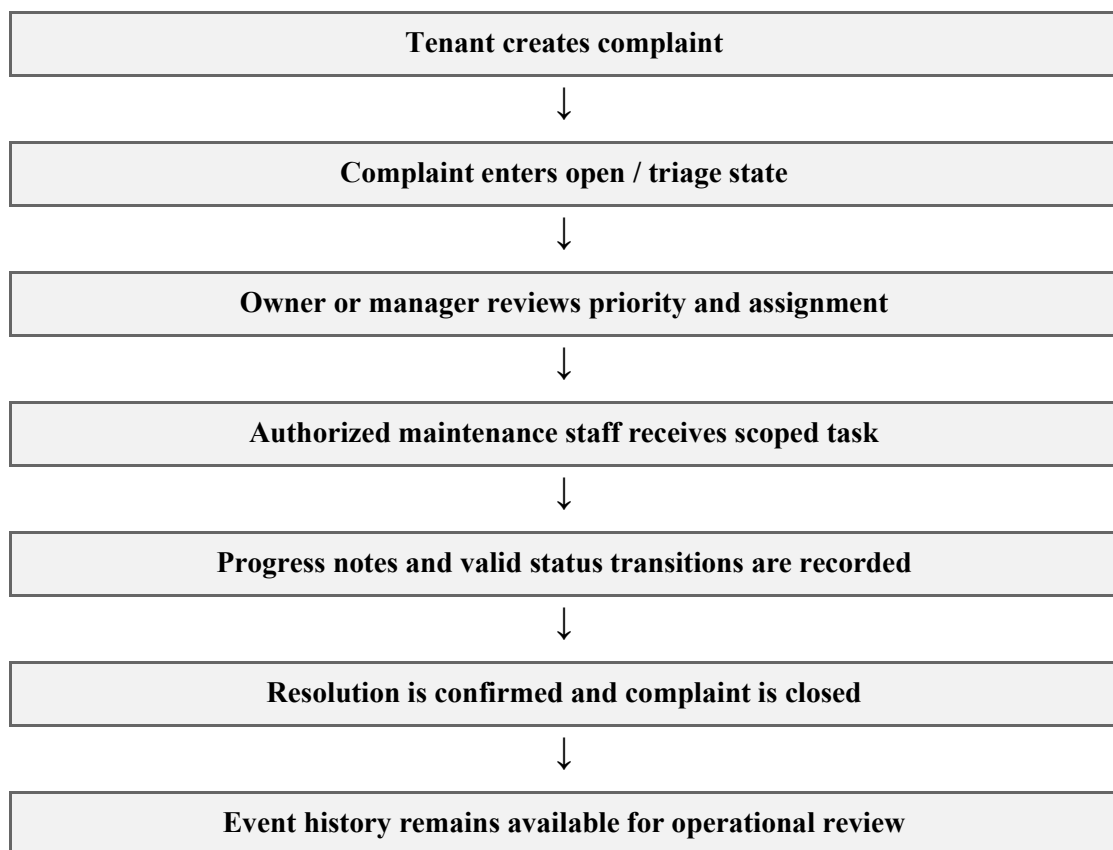


Figure 3.10. Complaint resolution and operational follow-up.

The same model supports accountability across roles. Tenants can follow the state of their request, operators can understand who handled it, and maintenance staff can focus on assigned work without receiving broad access to unrelated tenant or financial information. The event history also avoids relying on disconnected message threads as the only record of what happened.

CHAPTER 4

IMPLEMENTATION

4.1 Development Methodology

Development followed an iterative, backend-contract-first sequence. Shared routing, session restoration, theme tokens and test tooling established the client foundation. Timestamped migrations then introduced the schema, RLS policies, transactional functions, storage rules and release-readiness controls before dependent screens were completed. Later operational and invitation-validation migrations extended behavior without rewriting earlier migration history.

4.2 Technology Stack

The verified client stack uses Expo 56.0.20, React Native 0.85.3, React 19.2.3 and TypeScript 6.0.3. Expo Router 56.2.19 supplies file-based navigation; Supabase JS 2.110.8 provides backend access; TanStack Query 5.101.4 manages server state; React Hook Form 7.82.0 and Zod 4.4.3 support forms and validation; and Unwind, HeroUI Native, Lucide and Inter implement the interface system. Jest and pgTAP provide client and database-oriented verification.

4.3 Authentication and Protected Navigation

The root layout restores the Supabase session, initializes optional Sentry monitoring, registers push support when configured and routes users among public authentication, organization setup, invitation acceptance or a protected application area. Owner, tenant and staff route groups keep navigation task-focused. Unauthorized and configuration-required screens provide explicit fallbacks, while backend policies continue to decide whether data access is permitted.

4.4 Organization, Property and Occupancy Management

Owners create or select an organization before configuring properties, rooms and optional beds. Resident records can exist independently of an app account and can later be linked through a tenant invitation. Protected functions create tenancy and initial

occupancy together, validate availability and preserve assignment history during transfers. Current owner routes provide property, resident, tenancy, room and occupancy-management workflows.

4.5 Billing, Payments and Receipts

Billing supports invoice items, idempotent monthly generation, externally completed manual payments, tenant payment-proof submission, authorized decisions, allocations and receipts. Financial amounts are stored as integer paise. The client exposes invoice, payment and receipt routes, while protected database functions validate approved amounts and allocation boundaries before financial records are committed.

4.6 Complaints, Notices, Documents and Operations

Tenant complaint routes support creation and history; owner routes support triage and assignment; and staff routes expose assigned tasks. Notice services publish to approved targets and maintain recipient read state. Private documents and attachments use protected Storage paths. The operational extension adds expenses, agreement renewal, manual payment recording and a monthly profit report through database functions and corresponding owner screens.

4.7 Dashboards, Reports and Shared Components

Owner dashboards combine occupancy, collection, pending-decision and complaint information, while tenant dashboards emphasize dues, notices and requests and staff dashboards emphasize assigned work. Reports and CSV export reuse organization-scoped service data. Shared utilities centralize money, date, CSV and error behavior, and shared components provide consistent loading, empty, error and dialog states. Optional push and Sentry configuration are clearly separated from always-available application behavior.

4.8 Database Migrations and RLS

The current working tree contains six timestamped migrations and five pgTAP test files. The migrations cover the foundation schema, P0 operational model, a property-

return correction, release-readiness policies and RPCs, demonstration operations, and the create_invitation email-regex correction. Together they version tables, constraints, policies, storage rules, functions and grants so the client and backend can be deployed as a compatible contract.

The first migration establishes the foundation schema and the organization boundary used throughout the application. It creates the principal identity, membership, property, resident, occupancy, billing and operational relationships together with their initial constraints and access rules. The second migration extends this foundation with the P0 operational model needed by the client workflows. Keeping these definitions in ordered SQL files makes the backend contract reviewable: a fresh environment can reconstruct the same tables, keys, checks and policies without relying on undocumented dashboard edits.

The remaining migrations refine that contract in focused increments. A property-return correction aligns a database function with the value expected by the client; release-readiness SQL strengthens policies, storage behavior, functions and grants; demonstration operations introduce expenses, agreement renewal, manual payment recording and monthly profit reporting; and the final migration corrects invitation email validation. This sequence illustrates forward-only maintenance: each adjustment is visible as a new, timestamped change, while the history that produced an existing deployment remains intact and auditable.

RLS policy design follows the same scope model used by the application. Owner and manager access is derived from active organization or property membership, tenant access is connected to the authenticated user's resident and tenancy relationship, and maintenance access is limited to assigned operational work. These checks are applied when rows are selected or changed, independently of which screen issued the request. Consequently, role-specific routes improve usability, while PostgreSQL remains the authoritative control point for cross-organization isolation and record-level permission decisions.

Sensitive files use private Storage buckets and record-aware access policies. Payment evidence, complaint attachments, resident documents and expense receipts are stored by controlled object paths rather than exposed through permanent public URLs. Read decisions can therefore be related to the organization, tenancy, complaint or

expense record represented by the path and its associated database row. This arrangement keeps document handling consistent with the relational authorization model and enables time-limited access patterns without weakening the privacy of the underlying bucket.

Transactional functions concentrate multi-record state changes that must succeed as one unit. Tenancy creation coordinates the agreement and initial occupancy assignment; transfers preserve prior assignment history while checking destination availability; monthly invoice generation uses uniqueness and idempotency protections; and payment decisions validate amounts before allocations and receipt evidence are committed. Encapsulating these operations in protected functions reduces partial-update risk and gives the client a narrow domain-oriented interface instead of requiring a sequence of loosely coordinated writes.

4.9 Validation, Errors and Reliability

Client inputs use Zod and typed form handling, while PostgreSQL constraints remain authoritative. Query keys include user, organization and filter scope so cached records do not cross contexts. Errors are normalized into understandable messages, and financial or occupancy mutations wait for confirmed backend results before refreshing affected queries. Invitation validation is covered by the newest Jest suite and by the focused database migration.

Validation is deliberately layered. Forms provide immediate feedback through typed schemas, feature services normalize values before requests, and PostgreSQL constraints and protected functions make the final decision. Invitation creation is a useful example: the client trims and validates the address, confirms the requested role and property scope, and then invokes the database function; the newest migration corrects the function's email expression so server-side behavior matches the intended contract. This alignment improves consistency for invitations created from every supported client path.

Reliability after a mutation depends on confirmed state and selective refresh. Tenantly waits for authoritative completion of financial and occupancy operations, translates backend errors into understandable messages and invalidates query keys scoped to the affected user, organization and filters. The approach avoids displaying a

successful payment, allocation or room transfer before the database commits it. It also prevents a response cached for one organization from being reused in another active context, which is especially important on shared or long-running mobile sessions.

4.10 Authentication and Session Implementation

The authentication layer uses Supabase Auth to establish user identity and a session-aware root layout to decide which application area may be entered. Public authentication routes are kept separate from protected application routes. After authentication, the client resolves organization membership or invitation state before presenting owner, tenant or staff navigation. This keeps first-time setup and invitation acceptance distinct from routine operational use.

Session-aware navigation improves the user experience by preventing users from repeatedly entering screens they are not expected to use. However, the implementation treats navigation only as presentation logic. Data access continues to be authorized by Row Level Security and membership conditions in PostgreSQL, so possession of a route path alone is insufficient to retrieve protected records. This dual approach makes the application easier to use without weakening the database security boundary.

4.11 Property and Occupancy Implementation Details

The property module converts an organization into manageable physical inventory. Properties contain rooms and rooms may contain beds where shared occupancy is required. Resident records are maintained separately from user authentication so that an operator can prepare tenancy information before an invitation is accepted. This also reflects real operational practice, where a resident may be entered into the system before completing account setup.

Tenancy creation and occupancy assignment are handled as protected operations because they affect several related records and must remain consistent. Transfers close the prior active assignment and create a new assignment instead of deleting history. Move-out operations similarly preserve the earlier occupancy period. These choices support vacancy calculation, historical review and future reporting while reducing the risk of contradictory current-state records.

4.12 Billing and Payment Implementation Details

Billing is implemented around invoices, invoice line items, payments, allocations and receipts. The monthly generation process is designed so that retrying the same operation does not create repeated invoices for an eligible tenancy and period. This behavior is important for a mobile client because a user may retry an action after a slow network response even when the first request has already reached the backend.

Manual payment processing was retained in the MVP because it matches the project scope and allows payments made through external methods to be recorded without integrating a payment gateway. Tenants submit the amount, method, date, reference and supporting proof. An authorized operator reviews the evidence and makes the approval or rejection decision. Approved amounts are allocated through controlled backend logic, and the resulting receipt provides a stable record of the completed payment.

4.13 Complaints, Notices and Document Implementation Details

Operational features are built around role-appropriate visibility. Complaints retain an event history and controlled lifecycle; notices can be targeted to intended recipients while maintaining read state; and protected documents or attachments are referenced through private storage paths rather than permanent public URLs. These design choices reduce the need to distribute sensitive records through unrelated messaging applications.

Maintenance users receive a deliberately narrower operational view than owners or managers. This minimizes unnecessary exposure of financial or tenancy information while still allowing assigned work to progress. Shared error, loading and empty-state components provide consistent feedback across these modules, and domain-specific services keep data operations separate from screen layout code.

4.14 Reliability and Data-Integrity Implementation

Reliability is addressed at both client and database layers. Typed validation improves immediate feedback; relational constraints preserve valid values and relationships; and organization-scoped query keys prevent accidental cache reuse. Transactional RPCs coordinate multi-record operations, while idempotency and uniqueness controls protect

recurring billing and retry-sensitive mutations. Financial actions avoid optimistic confirmation where an uncommitted result could mislead the user.

Six ordered migrations reproduce the schema, policies, functions, storage configuration and grants, and generated TypeScript database types expose that contract to the client. This reduces dependence on manual backend configuration. The final invitation migration corrects email validation without altering prior migration files, demonstrating the project's forward-only approach to schema evolution.

CHAPTER 5

TESTING AND RESULTS

5.1 Testing Objectives and Environment

Verification covered type safety, domain services, shared UI state, formatting utilities, invitation validation and database-oriented policy artefacts. The current repository contains seventeen Jest suites and five Supabase pgTAP files. Commands were executed against the current working tree so the results in this chapter distinguish observed passes, actionable static-analysis findings, platform advisories and environment-dependent checks.

Verification is organized as a traceability chain from requirement to implementation artefact and observable evidence. Deterministic utilities and feature services are exercised by Jest; the compiler checks consistency across TypeScript boundaries; lint evaluates configured code-quality rules; Expo Doctor evaluates the project and dependency environment; and the pgTAP files express database expectations for schema, RLS and protected operations. Recording these evidence types separately makes the conclusions reproducible and prevents a configuration check from being confused with a functional assertion.

The repository also supports repeatable evaluation through named package scripts, versioned migrations and stable test locations. An assessor can install the declared dependencies, execute the client checks, provision the local Supabase services, apply the migration chain and run the five database files in their intended order. This is stronger than a screenshot-only demonstration because each result can be reproduced

from the same working tree and compared with the requirements, routes, services and SQL objects described in the preceding chapters.

5.2 Unit and Component Testing

The current Jest run completed successfully with 17 of 17 suites and 45 of 45 tests passing. Coverage includes money, date and CSV utilities; query keys and authentication types; error translation; shared screen and navigation states; organization, property, room, resident, billing, complaint and operations services; and invitation validation. No snapshots are used.

5.3 Static Analysis

The TypeScript command `tsc --noEmit` completed successfully. The configured Expo lint command completed its analysis and reported three actionable findings: one `react-hooks/set-state-in-effect` error and one `exhaustive-deps` warning in `src/app/(owner)/invite.tsx`, plus one `unused-import` warning in `src/shared/errors/to-user-message.test.ts`. These findings are recorded separately from the passing typecheck and Jest results so each quality signal remains precise.

5.4 Database and RLS Test Readiness

The repository provides five pgTAP files covering schema, RLS, the P0 model, release readiness and demonstration operations. The command `npm run supabase:test` reached the local-database connection stage but could not connect because a local PostgreSQL/Supabase stack was unavailable. The result is therefore classified as `environment-blocked`; the test artefacts remain ready for execution after local services start.

5.5 Expo Environment Check

A complete Expo Doctor run evaluated 22 checks and reported 21 passes. The single product advisory concerns the Hermes V1 version bundled with Expo 56.0.20 and React Native 0.85.3; Expo Doctor recommends Expo SDK 57 with expo 57.0.9 or later, or React Native 0.86.2 or later. Separate reruns also showed that online metadata

checks depend on network availability, which does not change the identified Hermes advisory.

5.6 Test Execution Summary

Table 5.1. Test execution summary.

Verification activity	Observed result	Status
Jest unit/component suite	17 suites passed; 45 tests passed.	PASS
TypeScript compilation	tsc --noEmit completed successfully.	PASS
Expo lint	Analysis completed with one error and two warnings.	OBSERVATIONS
Supabase pgTAP	Five files present; local database connection unavailable.	ENVIRONMENT BLOCKED
Expo Doctor	Complete run: 21/22 checks; Hermes upgrade advisory.	ADVISORY

5.7 Representative Test Cases

Table 5.2. Representative test cases.

ID	Scenario	Expected result	Evidence
TC-01	Money formatting	Indian currency values are formatted consistently.	Jest utility test
TC-02	Room behavior	Room data and validation are processed correctly.	Property/room tests
TC-03	Billing aggregates	Invoice and payment aggregates remain correct.	Billing tests
TC-04	Complaint behavior	Complaint service behavior is validated.	Complaint tests
TC-05	Operations services	Operational records and reports are processed correctly.	Operations tests
TC-06	Navigation states	Shared navigation and screen states render correctly.	Component tests
TC-07	Invitation validation	Email, role and property-scope rules are enforced.	Invitation service tests
TC-08	Database isolation	Policies and pgTAP artefacts cover scoped access.	Migrations / pgTAP

5.7.1 Test Strategy by Risk Area

The test strategy gives priority to money, occupancy, authorization, validation and retry-sensitive behavior. Utility tests cover deterministic formatting and cache-key behavior; feature-service tests cover organizations, invitations, properties, residents, billing, complaints and operations; and component tests cover shared screen and navigation states. TypeScript and lint add independent static-analysis signals, while pgTAP files target database constraints, functions and RLS.

Risk-based coverage gives priority to operations where an incorrect result would damage financial evidence, occupancy history or privacy. Billing tests examine calculations and service aggregation; property, room and resident tests exercise core inventory behavior; invitation tests cover identity and scope validation; and shared component tests confirm consistent loading, empty, error and navigation states. Database-oriented cases complement these checks by specifying the authorization and transaction behavior that must hold when the migrations are exercised against a provisioned PostgreSQL environment.

Each representative case identifies the condition being evaluated, the expected safe outcome and the artefact that provides evidence. This format supports bidirectional review: a requirement can be followed forward to its implementation and test, while an individual test can be followed backward to the risk it addresses. The extended matrix applies the same method to authentication, organization isolation, occupancy conflicts, retry behavior, payment approval, private files, complaints and role-scoped reporting, producing a coherent verification narrative rather than an unrelated list of commands.

The 45 passing Jest tests across 17 suites are executed evidence for the covered client and service behavior, and the successful TypeScript run is executed evidence of type consistency. Lint completed analysis and identified three focused findings rather than producing a clean pass. The five database files demonstrate test readiness but are not counted as executed passes because the local database connection was unavailable.

5.7.2 Functional Validation Scenarios

Property and resident scenarios focus on whether structured records can be processed without losing the organization context required by later operations. Billing scenarios focus on totals, currency representation and aggregation behavior because financial

values are reused in dashboards and reports. Complaint and operations scenarios verify domain-service behavior around operational records. Shared-state tests check loading, empty, error and navigation behavior so that common interface conditions do not depend on each feature implementing its own inconsistent handling.

Several important risks are addressed primarily through the database design rather than by the Jest suite alone. Conflicting occupancy is mitigated through transaction-oriented assignment and transfer functions. Duplicate monthly billing is mitigated by idempotent generation and uniqueness controls. Unauthorized access is addressed through RLS policies and membership scope. Payment over-allocation and receipt integrity are handled by guarded financial operations. These mechanisms are documented as implementation controls; the dedicated database tests remain the correct place to execute direct policy and constraint verification once the local Supabase environment is available.

5.7.3 Reliability, Negative Cases and Recovery

Mobile clients must handle repeated taps, uncertain connectivity and data that changes after a screen loads. Tenantly keeps authoritative validation in PostgreSQL, scopes query keys to the active context, refreshes affected data after mutations and avoids optimistic confirmation for high-risk finance operations. Idempotency keys, uniqueness rules and transactional functions reduce the chance that a retry creates a duplicate invoice, assignment or decision.

Expo Doctor's Hermes result is recorded as a dependency-baseline advisory, distinct from functional test outcomes. The recommended upgrade path is explicit and can be evaluated as part of release maintenance. Network-dependent Doctor checks are also recognized as environment-sensitive, while the stable repository evidence remains the passing Jest and TypeScript results plus the focused lint findings.

5.7.4 Interpreting the Verification Evidence

The evidence supports three clear conclusions: the implemented Jest suite passes, the current TypeScript code is type-consistent, and the repository contains a substantial database verification suite. Lint supplied three targeted code-quality observations, while pgTAP execution requires the local Supabase stack. This classification avoids

treating a test file, a successful command, an advisory and an environment block as equivalent forms of evidence.

5.8 Defect and Risk Resolution

The implementation addresses high-risk cases through explicit controls: protected occupancy functions preserve history and availability, invoice generation is idempotent, payment decisions and allocations are backend-controlled, receipts are protected from ordinary modification, and RLS limits organization and tenant visibility. Current verification additionally provides passing client tests, a passing typecheck and focused follow-up observations from lint and Expo Doctor.

5.9 Extended Functional and Security Verification Matrix

The matrix maps major Tenantly workflows to their verification condition and expected safe behavior. It serves as a traceability aid: rows backed by Jest or static analysis reflect executed client evidence, while authorization, transaction and storage rows identify behavior designed for pgTAP execution against a running Supabase environment. Environment-blocked database tests are not converted into claimed passes.

ID	Area	Verification condition	Expected safe behavior
EV-01	Authentication	Valid credentials establish a session and protected navigation becomes available.	Session restored; role routing proceeds.
EV-02	Authentication	Unauthenticated access to protected data is attempted.	Request is denied / no protected data returned.
EV-03	Organization isolation	A user attempts to access a record belonging to another organization.	RLS prevents cross-organization visibility.
EV-04	Property setup	Owner creates property and room inventory with valid values.	Records are created within authorized organization scope.
EV-05	Occupancy	Assignment targets an already occupied constrained room or bed.	Protected operation prevents conflicting active occupancy.
EV-06	Transfer	Resident is transferred from one active assignment to another.	Prior assignment closes and new assignment preserves history.
EV-07	Billing	Monthly invoice generation is retried for the same eligible period.	Idempotent behavior avoids duplicate invoices.
EV-08	Payment	Tenant submits payment evidence for an invoice.	Submission is recorded for authorized review.
EV-09	Payment decision	Unauthorized role attempts to approve or reject a payment.	Decision operation is denied.

ID	Area	Verification condition	Expected safe behavior
EV-10	Receipt	Approved payment completes the receipt-producing workflow.	Receipt remains stable as financial evidence.

Table 5.3. Extended verification matrix - identity, occupancy and finance.

ID	Area	Verification condition	Expected safe behavior
EV-11	Complaints	Tenant creates a complaint with required details.	Request is recorded and visible within permitted scope.
EV-12	Complaints	A role attempts an invalid complaint transition.	Transition function rejects the invalid state change.
EV-13	Maintenance	Staff member opens assigned operational work.	Only authorized task scope is available.
EV-14	Notices	Operator publishes a notice to an approved target.	Intended recipients can access the notice and read state.
EV-15	Documents	User requests a private file outside authorized scope.	Storage/database policy prevents access.
EV-16	Validation	Form contains an invalid or incomplete domain value.	Client validation gives feedback and backend constraints remain authoritative.
EV-17	Network retry	A mutation is retried after an uncertain mobile response.	Transaction/idempotency controls avoid duplicate high-risk results.
EV-18	Static analysis	TypeScript source is checked without emission.	Type consistency check completes successfully as reported.
EV-19	Lint	Configured Expo lint command is executed with a zero-warning threshold.	Analysis reports findings against the configured threshold.
EV-20	Database suite readiness	Supabase test suite is invoked without a running local stack.	Environment block is reported, not mislabeled as a pass.

Table 5.4. Extended verification matrix - operations, validation and tooling.

The matrix deliberately separates expected system behavior from observed tool execution. Authorization, transaction and storage scenarios should be executed after a local or staging Supabase environment is available. The present evidence remains transparent: Jest and TypeScript passed, lint produced focused findings, Expo Doctor produced a platform advisory, and the database command was blocked at connection time.

CHAPTER 6

CONCLUSION

6.1 Summary of Implementation

Tenantly delivers a mobile-first property-management MVP covering physical inventory, residents, tenancy and occupancy history, invoices, manual payment evidence, complaints, notices, private documents, operational expenses and reporting. The implementation emphasizes protected workflows, traceable records and organization isolation rather than interface breadth alone.

6.2 Achievement of Objectives

The project establishes role-aware application areas, organization-scoped data, structured occupancy, auditable billing and payments, complaint lifecycle management, targeted communication and repository-based verification. Sensitive multi-record actions are implemented through database functions and reinforced by RLS and private Storage policies. The current client test suite and typecheck provide executed evidence for the implemented code paths they cover.

6.3 Advantages

Tenantly improves operational clarity by preserving one source of truth for properties, residents and money. It supports tenant self-service, lowers reliance on ad-hoc message threads, records decision history and uses consistent organization boundaries. The shared Expo codebase also supports a common workflow across Android, iOS and web.

6.4 Current Limitations

The MVP intentionally excludes an online payment gateway and public rental marketplace. Database and RLS execution depends on an available local or staging Supabase stack. The current dependency baseline also carries the Expo Doctor Hermes memory advisory, and lint has identified three contained code-quality observations.

These boundaries are documented so pilot and release decisions can use an accurate verification picture.

6.5 Future Scope

Future development can add payment-gateway integration, scheduled reminders, richer analytics, offline draft and queue behavior, agreement automation, messaging integrations and public discovery. Release maintenance can also evaluate the Expo SDK 57 upgrade recommended by Expo Doctor and add routine CI execution of lint, migration reset and pgTAP checks against a provisioned database environment.

REFERENCES

Expo. (2026). Core concepts. <https://docs.expo.dev/core-concepts/>

Expo. (2026). Expo documentation. <https://docs.expo.dev/>

Meta. (2026). React Native documentation. <https://reactnative.dev/docs/getting-started>

Supabase. (2026). Database overview. <https://supabase.com/docs/guides/database/overview>

Supabase. (2026). Row Level Security. <https://supabase.com/docs/guides/database/postgres/row-level-security>

Supabase. (2026). Securing your API. <https://supabase.com/docs/guides/api/securing-your-api>

Supabase. (2026). Production checklist. <https://supabase.com/docs/guides/deployment/going-into-prod>

PostgreSQL Global Development Group. (2026). PostgreSQL documentation. <https://www.postgresql.org/docs/>

TypeScript. (2026). TypeScript handbook. <https://www.typescriptlang.org/docs/handbook/intro.html>

TanStack. (2026). TanStack Query documentation. <https://tanstack.com/query/latest>

Zod. (2026). Zod documentation. <https://zod.dev/>

Jest. (2026). Jest documentation. <https://jestjs.io/docs/getting-started>

APPENDICES

Appendix A: Installation and Configuration

Install dependencies with `npm install`, copy `.env.example` to `.env`, and provide the public Supabase URL and publishable key. Apply all six migrations in timestamp order. Never place a service-role key in an `EXPO_PUBLIC` value. For local verification, start Docker Desktop, run `npm run supabase:start` and `npm run supabase:reset`, execute `npm run supabase:test`, and regenerate client types with `npm run supabase:types`. Push registration additionally requires `EXPO_PUBLIC_EAS_PROJECT_ID`; Sentry remains optional through `EXPO_PUBLIC_SENTRY_DSN`.

A controlled deployment begins by applying the six migrations to the target Supabase project in timestamp order and confirming that public client configuration contains only the project URL and publishable key. The Expo application can then be built from the matching repository revision and exercised with owner, tenant and staff accounts. A practical acceptance sequence covers organization setup, property and room creation, resident invitation, tenancy assignment, invoice generation, payment-proof submission, approval, receipt access, complaint assignment and the monthly profit report.

Operational readiness is strengthened by retaining migration output, verification logs and the application revision used for each release. Administrators should review membership and property scope before onboarding users, use the protected workflows for finance and occupancy changes, and preserve completed receipts and assignment history. Optional push notifications and Sentry monitoring can be enabled through their documented public configuration values when required, while the core authenticated workflows remain based on Supabase Auth, PostgreSQL, RLS and private Storage.

Configuration Checklist

- Provide `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY` in the application environment.
- Keep service-role credentials in server-side Supabase secrets only.

- Apply migrations in timestamp order and deploy a compatible client and database change together.
- Confirm private Storage buckets, RLS policies and allowed file types before operational use.
- Use separate local, staging and production projects to avoid testing against production data.

Appendix B: Basic User Manual

Owner and Manager Workflow

- Register or sign in, then create or select an organization.
- Create a property and configure rooms; add beds where shared occupancy is required.
- Create resident records and tenancy terms, then assign a room or bed.
- Generate monthly invoices for active tenancies and review the invoice ledger.
- Review submitted payment proof, approve or reject it with an appropriate reason, and retain the resulting receipt.
- Use complaints, notices, documents and reports to manage continuing operations.

Tenant Workflow

- Sign in through the invitation-linked account and open the tenant dashboard.
- Review the current invoice and past payment state.
- Submit manual payment proof with payment method, date, reference, amount and attachment.
- Open a complaint with category, priority and a clear description; follow the status history.
- Read targeted notices and access authorized documents from the More area.

Maintenance Staff Workflow

- Sign in using the assigned staff membership.
- Open the task list to view only authorized requests.
- Add progress notes and move a request through the allowed lifecycle.
- Coordinate resolution through the recorded complaint history rather than separate message threads.

Detailed Owner / Manager Operating Sequence

A typical owner or manager begins by authenticating and selecting or creating the authorized organization. Property setup then establishes the physical inventory: the property, its rooms and any bed-level units required for shared accommodation. Resident records can be created before occupancy is finalized, allowing identity and contact information to exist independently of the tenancy. The manager then records tenancy terms and assigns the resident to an available room or bed through the protected occupancy workflow. If the resident later moves, the transfer process should be used rather than editing the historical assignment directly.

During each billing period, the manager generates invoices for eligible active tenancies and reviews the resulting ledger. When a resident submits manual payment evidence, an authorized operator reviews the amount, method, date, reference and attachment before approving or rejecting the submission. Approval creates the appropriate financial allocation while preserving the submitted evidence and decision history. The owner dashboard can then be used to review occupancy, collections, pending payment decisions and unresolved complaints. Notices, documents, expenses and agreement-related operations are handled from their respective modules rather than being mixed into the financial ledger.

Detailed Tenant Operating Sequence

A tenant enters the system through an account connected to an authorized invitation and uses the tenant-specific application area. The dashboard emphasizes information directly related to the tenant, such as current dues, notices and active requests. For rent, the tenant reviews the invoice before submitting payment evidence. The submission records the payment method, date, reference, amount and attachment required by the workflow. The payment remains subject to authorized review, so the tenant-facing state can distinguish a submitted payment from an approved financial record.

For maintenance or service issues, the tenant creates a complaint with an appropriate category, priority and description. The complaint history provides a structured record of progress instead of requiring the tenant to reconstruct updates from separate messages. Targeted notices appear in the tenant area, and authorized documents are accessed through protected storage rather than public file links. The role-focused

interface is intended to make these routine tasks available without exposing administrative property-management controls.

Detailed Maintenance Staff Operating Sequence

Maintenance staff authenticate through their assigned organization membership and use the staff task area. Their access is intentionally narrower than owner or manager access. Assigned requests can be reviewed with the information needed to perform the work, and progress notes or permitted state transitions can be recorded as the task develops. This keeps maintenance updates attached to the complaint lifecycle and allows owners, managers and the requesting tenant to rely on the same recorded history rather than separate status conversations.

Operational Good Practices

Users should avoid sharing accounts because authorization and auditability depend on authenticated identity and membership. Owners should complete room or bed transfers through the designated occupancy workflow so historical assignments remain intact. Payment evidence should be reviewed before approval, and completed financial records should not be replaced with informal notes. Sensitive attachments should remain inside the protected document and evidence workflows. Administrators should also keep the application and Supabase migrations compatible, verify private storage policies after deployment changes and execute the configured database test suite whenever a local or staging Supabase environment is available.

Appendix C: Important Database Functions

Function	Purpose
<code>create_organization_with_owner</code>	Creates the initial organization and owner membership.
<code>create_tenancy_with_assignment</code>	Creates tenancy and occupancy assignment in one operation.
<code>transfer_occupancy</code>	Closes prior assignment and creates a new active state.
<code>generate_monthly_invoices</code>	Creates missing invoices for eligible tenancies.
<code>submit_payment / decide_payment</code>	Submits payment evidence and performs authorized decision/allocation.
<code>transition_complaint</code>	Records validated complaint lifecycle changes.
<code>create_invitation / accept_invitation</code>	Issues and accepts controlled membership invitations.

Function	Purpose
publish_notice	Creates notices and their approved targets.
owner_dashboard / monthly_profit_report	Returns organization-scoped dashboard/report data.

Data Integrity Controls

Primary keys, foreign keys, uniqueness rules, state checks, RLS policies, private Storage policies and protected functions work together to preserve integrity. Money is stored as integer paise, occupancy uses time-aware assignments, recurring invoices use uniqueness and idempotency controls, and completed financial evidence is protected from ordinary mutation. These controls make the operational model auditable while keeping authorization in the backend.

Appendix D: Extended Test-Case Matrix

ID	Test scenario	Expected outcome	Verification layer
TC-09	Email login with valid credentials	Session is restored and protected routes become available.	Manual / Auth flow
TC-10	Password reset request	Recovery flow accepts the email and provides safe feedback.	Manual / Auth route
TC-11	Organization creation	Initial owner membership is created with the organization.	Database function
TC-12	Property creation by owner	Authorized operator can create property under the active organization.	RLS / service
TC-13	Room creation with beds	Room and optional bed inventory are created together.	Database function
TC-14	Resident created before app account	Resident record can exist without linked profile.	Schema / service
TC-15	Tenancy and initial assignment	Tenancy and occupancy state are created atomically.	Database function
TC-16	Room transfer	Prior occupancy closes and new occupancy becomes active.	Database function
TC-17	Overlapping bed occupancy	Conflicting active assignment is rejected.	Constraint / function
TC-18	Monthly invoice rerun	Existing invoice is skipped rather than duplicated.	Database function
TC-19	Partial payment	Allocated amount updates balance without exceeding payment.	Billing logic
TC-20	Unauthorized payment decision	Tenant cannot approve a submitted payment.	RLS / function
TC-21	Receipt protection	Immutable receipt cannot be silently modified.	Trigger / schema
TC-22	Tenant complaint creation	Linked tenant may create a scoped complaint.	RLS / function
TC-23	Invalid complaint transition	Disallowed status update is rejected.	Database function
TC-24	Targeted notice read	Recipient can read intended notice and update own read state.	RLS / service

Appendix D Continued: Extended Test-Case Matrix

ID	Test scenario	Expected outcome	Verification layer
TC-25	Private document access	Non-authorized users cannot read sensitive storage object.	Storage policy
TC-26	Complaint attachment	Attachment path is associated with authorized complaint record.	Storage / function
TC-27	CSV export	Export preserves expected invoice/payment columns.	Jest utility test
TC-28	Date formatting	Local date helper formats values consistently.	Jest utility test
TC-29	Money formatting	Paise values are rendered consistently for INR display.	Jest utility test
TC-30	Query-key scoping	Organization/filter scope is included in query identity.	Jest utility test
TC-31	Error translation	Database/application failures map to understandable user messages.	Jest unit test
TC-32	Loading and empty states	Shared state views render the appropriate screen state.	Component test
TC-33	Navigation guards	Navigation helpers choose the correct safe route.	Component test
TC-34	Expense recording	Authorized operational expense is persisted and reportable.	Database function
TC-35	Agreement renewal	Authorized renewal updates document/tenancy operational state.	Database function
TC-36	Monthly profit report	Organization-scoped finance summary is returned.	Database function

Appendix E: Verification Commands

- `npm test` - Runs the Jest suite; the current run passes 17 suites and 45 tests.
- `npm run typecheck` - Validates TypeScript without emitting files; the current run passes.
- `npm run lint` - Runs configured Expo lint analysis and reports actionable findings.
- `npm run supabase:start`, then `npm run supabase:test` - Starts local services and executes five pgTAP files.
- `npm run doctor -- --verbose` - Runs Expo environment and dependency checks.

Appendix F: Role and Data-Access Matrix

This appendix summarizes intended role separation without replacing the deployed authorization rules. Actual access depends on authenticated identity, active organization and property membership, tenant/resident relationships, RLS policies and protected functions. The matrix therefore describes practical scope while the database remains authoritative.

Resource / action	Owner	Manager	Tenant	Maintenance staff
Organization settings	Create/manage within authorized scope	Limited operational access where granted	No	No
Property / room / bed inventory	Create and manage	Manage when permitted	View tenancy-related information only	Assigned task context only
Residents and tenancies	Create/manage	Manage when permitted	Own tenancy only	No general resident access
Invoices	Generate/view/manage	Operational access when permitted	Own invoices	No
Payment proof	Review and decide	Review/decide when authorized	Submit and view own	No
Complaints	Triage/assign/transition	Triage/assign/transition when permitted	Create and view own	View/update assigned work
Notices	Publish to approved targets	Publish when permitted	Read targeted notices	Read targeted/operational notices if authorized
Private documents	Access within organization policy	Access when permitted	Own/authorized documents	Only explicitly authorized files
Reports / dashboards	Organization operational view	Scoped operational view	Personal dashboard	Task-focused dashboard

Table F.1. Role and data-access summary.

Authorization interpretation

Terms such as 'when permitted' are intentional because a role label does not grant universal access. A manager's property scope is evaluated inside the authorized organization; a tenant is limited to records connected to the tenant's own resident or tenancy relationship; and maintenance staff receive assigned operational work rather than broad financial access. Route visibility supports usability, while RLS and RPC checks enforce access.

Appendix G: Data Model Traceability

The following traceability summary connects the major entity groups to the business purpose and integrity concern they address. It consolidates information already described in the system-design and implementation chapters and can be used by an evaluator to understand why the schema is divided into multiple related records instead of one large property table.

Entity group	Business purpose	Primary design / integrity rationale
Profiles / organizations / memberships	Identity and organizational scope	Links authenticated users to the business boundary used by authorization policies.
Properties / rooms / beds	Physical rental inventory	Separates property structure from resident and contract records; supports vacancy derivation.
Residents / tenancies	Resident identity and contractual occupancy terms	Allows a resident record to exist independently of authentication and preserves tenancy context.
Occupancy assignments	Time-aware room/bed occupancy history	Prevents current-state overwrite from destroying transfer and move-out history.
Invoices / invoice items	Billing obligation for a defined period	Keeps billed amounts and line-item composition distinct from later payment events.
Payments / allocations / receipts	Submitted money, approved application and evidence	Supports partial/multiple allocations and auditable receipt generation.
Complaints / complaint events	Operational request lifecycle	Preserves priority, assignment and chronological state-transition history.
Notices / targets / read state	Controlled communication	Separates publication, audience targeting and recipient-level read tracking.
Documents / attachments	Protected file metadata	Avoids treating sensitive files as unrestricted public objects.
Audit / notification / device records	Operational traceability and delivery support	Supports historical review and system communication without mixing these concerns into core entities.

Table G.1. Major entity groups and their design rationale.

Relationship interpretation

The strongest relationships in the model correspond to recurring operational workflows. Organization membership controls who may work with a property; a property contains room or bed inventory; a tenancy connects a resident to contractual terms; occupancy assignments represent the time-bounded physical placement; invoices represent periodic obligations; payments and allocations describe how money is applied; and

receipts provide completed financial evidence. Keeping these concepts separate allows each workflow to evolve without losing historical meaning.

This decomposition also supports safer database constraints. Uniqueness and foreign-key rules can be applied at the entity where they are meaningful, while protected functions can coordinate changes that span multiple tables. In practical terms, the model is intended to make contradictory active occupancy, duplicate recurring invoices and unauthorized financial decisions harder to create than they would be in a spreadsheet-style single-record design.

Schema Evolution Considerations

Tenantly uses six ordered migrations so schema changes travel with their policies, functions and grants. Client features should be deployed with the migration versions they require rather than with manual production edits. The final migration demonstrates this approach by correcting `create_invitation` email validation as a new forward-only change.

Future migrations should preserve completed financial evidence and occupancy history wherever possible. New reporting fields or operational modules can extend the schema without rewriting receipts or deleting earlier assignments. This approach retains traceability while allowing the backend contract to evolve alongside compatible client releases.